Typesetting Formal Semantics in Typst

A Guide from Document Basics to Lambda Calculus

Utku Turk

University of Maryland, College Park

1 Why Typst?

If you have used LaTeX, you know the loop: edit, compile, wait, check the PDF, fix one thing, compile again. Typst is much faster in actual use because the preview updates incrementally while you type. For a semantics paper with trees, glosses, and derivations, that matters.

The second advantage is that Typst reads more like code you actually want to maintain. Instead of piling macros into a preamble, you can write small functions for repeated jobs: semantic brackets, citation helpers, jamboxes, derivation layouts, or different header behavior on page 1 versus later pages.

For example, this document's header logic is ordinary Typst code:

Code

```
#set page(
  header: context {
    if counter(page).get().first() > 1 {
      grid(
        columns: (1fr, 1fr),
        align(left)[Typst for Linguists],
        align(right)[Utku Turk],
      )
    }
  },
)
```

Output

Pages after the first get a running header; page 1 does not.

The syntax is also more transparent. Where LaTeX needs `\textbf{bold}`, Typst uses `*bold*`. Where LaTeX needs `\begin{itemize} \item ... \end{itemize}`, Typst uses `- item`. If you can read `#text(size: 14pt)[Hello]`, you already understand the basic shape of the language.

The ecosystem is younger, so some linguistics workflows still need manual setup. That is what this guide is for.

2 Typst Basics

2.1 Document Metadata, Pages, and Fonts

Code

```
#set document(title: "My Paper", author: "Your Name")
#set page(paper: "us-letter", margin: (x: 1in, y: 1in))
#set text(font: "Charis SIL", size: 11pt, lang: "en")
#set par(justify: true, first-line-indent: 1.2em, leading: 0.65em)
#set heading(numbering: "1.1")
```

Output

My Paper
Your Name

Other page options: `"a4"`, per-side margins with `margin: (top: 1in, bottom: 1in, left: 1.5in, right: 1in)`.

2.2 Headings

`= Top-Level`, `== Second Level`, `=== Third Level`. Add numbering with `#set heading(numbering: "1.1")`.

2.3 Inline Formatting

`*bold*` → **bold**, `_italic_` → *italic*, `#underline[text]` → text, `#strike[text]` → ~~text~~, `#smallcaps[text]` → TEXT. Lists: `-` for bullets, `+` for numbered.

2.4 Figures and Tables

Reference with `@tab:types`. This produces Table 1:

Code

```
#figure(
  table(
    columns: (auto, auto, auto),
    align: (left, left, left),
    stroke: none,
    inset: 5pt,
    table.header[*Expression*][*Type*][*Gloss*],
    table.hline(stroke: 0.4pt),
    [_Mary_],   [$e$],                          [individual],
    [_runs_],   [$e -> t$],                     [one-place predicate],
    [_loves_],  [$e -> (e -> t)$],              [two-place relation],
    [_every_],  [$(e -> t) -> ((e -> t) -> t)$],    [GQ det],
  ),
  caption: [Lexical types for selected English expressions.],
) <tab:types>
```

Output

Expression	Type	Gloss
Mary	e	individual
runs	$e \to t$	one-place predicate
loves	$e \to (e \to t)$	two-place relation
every	$(e \to t) \to ((e \to t) \to t)$	GQ det

Table 1: Lexical types for selected English expressions.

2.5 Boxes and Colored Boxes

A framed box: `#block(stroke: 0.5pt, inset: 1em, radius: 3pt)[...]`:

Code

```
#block(
  width: 100%, stroke: 0.5pt + luma(120),
  inset: 1em, radius: 3pt,
)[Content here.]

#let tip(body) = block(
  fill: rgb("#f0f7ff"),
  stroke: (left: 3pt + rgb("#4a90d9")),
  inset: (x: 1em, y: 0.7em), radius: 2pt,
)[*Tip:* #body]
```

Output

Content here.

Tip: Wrap the pattern in a function and reuse it.

2.6 Cross-References and Bibliography

Label with `<label>` after any heading, figure, or example. Reference with `@label`. For bibliography:

```
#bibliography("refs.bib", style: "chicago-author-date")
```

Cite with `@citekey` for parenthetical or `#cite(<citekey>, form: "prose")` for “Author (Year)”. Rather than typing `form: "prose"` every time, define short helpers:

```
// Author (Year) — like \citet or \textcite
#let narr(key) = cite(key, form: "prose")

// Author's (Year) — like \citeauthor's \citeyearpar
#let gen(key) = {
  cite(key, form: "author")
  [\u{2019}s ]
  cite(key, form: "year")
}
```

Then `#narr(<rooth1992>)` gives "Rooth (1992)" and `#gen(<atlamaz2023>)` gives "Atlamaz's (2023)".

3 Linguistic Examples with `eggs`

The `eggs` package handles numbered examples, sub-examples, grammaticality judgments, glossing, and cross-referencing. Import it and apply the show rule at the top of your file:

```
#import "@preview/eggs:0.6.0": *
#import abbreviations: nom, acc, pst, q
#show: eggs
```

3.1 Basic Examples

Inside `#example[...]`, content is a single example. Numbered lists (`+`) become sub-examples automatically:

Code

```
#example[
  + Heidi ate the döner.
  + Heidi ate the D#smallcaps[öner]#sub[F].
  #ex-label(<donerpair>)
]
```

Output

(1) a. Heidi ate the döner.
b. Heidi ate the DÖNER$_{F}$.

3.2 Grammaticality Judgments

Prefix sub-examples with `\*`, `\#`, `\?`, or `\%` and `eggs` handles them automatically, pulling the judge mark into the left margin:

Code

```
#example[
  + Heidi ate the döner.
  + \*Ate Heidi döner the.
  + \#Heidi ate the döner, but I don't care if Bill ate it.
  + \?The döner Heidi ate.
  + \%Heidi the döner ate.
]
```

Output

(2) a. Heidi ate the döner.
b. *Ate Heidi döner the.
c. #Heidi ate the döner, but I don't care if Bill ate it.
d. ?The döner Heidi ate.
e. %Heidi the döner ate.

For custom judgments (e.g., `??` or `#?`), use `#judge[??]` inside the sub-example.

3.3 Cross-Referencing Examples

Label with `#ex-label(<name>)` inside the example. Reference with `@name` or for more control use `#ex-ref()`:

```
#ex-ref(<donerpair>)                    // (1)
@donerpair                              // (1)
#ex-ref(<donerpair:a>)                  // (1a)
#ex-ref(<donerpair:a>, <donerpair:b>)   // (1a--b)
#ex-ref(1)                     // relative: next example
```

3.4 Interlinear Glosses

Inside `#example[...]`, bullet lists (`-`) are treated as glosses. Separate words with **two or more spaces**:

Code

```
#example[
  - Heidi  döner-i       ye-di=mi?
  - Heidi  döner-#acc    eat-#pst.3#smallcaps[sg]=#q
  'Did Heidi eat the döner?'
  #ex-label(<pq>)
]
```

Output

(3) Heidi döner-i ye-di=mi?
Heidi döner-ACC eat-PST.3SG=Q
'Did Heidi eat the döner?'

Leipzig abbreviations imported from the `abbreviations` submodule auto-render in small caps. Print the full list with `#print-abbreviations()`.

3.5 Jambox-Style Annotations

`eggs` does not have a built-in jambox. A small helper does the job:

```
#let jb(main, annot) = grid(
  columns: (1fr, auto),
  main,
  align(right, text(size: 9pt, fill: luma(100))[#annot]),
)
```

Code

```
#example[
  + #jb[H#smallcaps[EIDI]#sub[F]=mi döneri yedi?][Subject NFQ]
  + #jb[Heidi D#smallcaps[ÖNER]-i#sub[F]=mi yedi?][Object NFQ]
  + #jb[Heidi döneri BUG#smallcaps[ÜN]#sub[F]=mü yedi?][Adjunct NFQ]
  + #jb[Heidi döneri Y#smallcaps[E]-D#smallcaps[İ]#sub[F]=mi?][Verum/TAM NFQ]
]
```

Output

(4) a. HEIDI$_F$ = mi döneri yedi? — Subject NFQ
b. Heidi DÖNER-i$_F$ = mi yedi? — Object NFQ
c. Heidi döneri BUGÜN$_F$ = mü yedi? — Adjunct NFQ
d. Heidi döneri YE-DI$_F$ = mi? — Verum/TAM NFQ

4 Formal Semantics Notation

No `\usepackage{stmaryrd}` needed. Everything lives in math mode.

4.1 Denotation Brackets

Code

```
#let sem(x) = $lr(bracket.l.stroked #x bracket.r.stroked)$
```

Output

```
$sem("runs")$ -> ⟦runs⟧
```

The `lr()` makes brackets scale.

> **Warning:** `bracket.l.double` / `bracket.r.double` are **deprecated**. Use `bracket.l.stroked` / `bracket.r.stroked`. Do not paste CJK brackets 〚〛 (U+301A/B) — they are not in Typst's math delimiter set.

4.2 Ordinary / Focus Denotations

```
#let semo(x) = $lr(bracket.l.stroked #x bracket.r.stroked)^o$
#let semf(x) = $lr(bracket.l.stroked #x bracket.r.stroked)^f$
```

$[\![\text{TP}]\!]^o$ and $[\![\text{TP}]\!]^f$. World-indexed: $[\![\text{runs}]\!]^w$.

Code

```
#example[$semo("TP") = "ate"("heidi","doner")$]
#example[$semf("TP") = mat(delim: "{",
  "ate"("heidi","doner") ,;
  "ate"("heidi","dolma") ,;
  "ate"("heidi","donut") ,;
  dots.v;
)$]
```

Output

(5) $[\![\text{TP}]\!]^o = \text{ate}(\text{heidi}, \text{doner})$

(6) $[\![\text{TP}]\!]^f = \left\{ \begin{matrix} \text{ate}(\text{heidi},\text{doner}) \\ \text{ate}(\text{heidi},\text{dolma}) \\ \text{ate}(\text{heidi},\text{donut}) \\ \vdots \end{matrix} \right\}$

4.3 Lambda Expressions

```
#let lam(v, body) = $lambda #v . thin #body$
```

$\lambda\ \mathrm{x}\ .\ \varphi(x) \cdot \lambda\ \mathrm{y}\ .\ \lambda\ \mathrm{x}\ .\ \mathrm{love}(\mathrm{x}, \mathrm{y})$

4.4 Semantic Types

Arrow: `$e -> t$` $\to e \to t$. Angle brackets:

```
#let ty(..args) = {
  let a = args.pos()
  $angle.l #a.join($, $) angle.r$
}
```

$\langle e, t\rangle \cdot \langle e, \langle s, t\rangle\rangle$ — replaces your LaTeX `\type` command.

4.5 Quantifiers and Logical Symbols

All built in: `forall` $\forall$, `exists` $\exists$, `not` $\neg$, `and` $\wedge$, `or` $\vee$, `->` $\to$, `<->` $\leftrightarrow$, `~>` $\rightsquigarrow$, `=>` $\Rightarrow$, `mapsto` $\mapsto$, `square` $\Box$, `diamond.stroked` $\diamond$, `iota` ι, `models` $\models$.

4.6 Semantics Inside Examples

Denotation brackets work directly inside `#example[...]`:

(7) $[\![C_{+\mathrm{pq}}]\!] = \lambda\ \mathrm{p}\ .\ \lambda\ \mathrm{q}\ .\ \mathrm{q} = \mathrm{p} \vee \mathrm{q} = \neg\ \mathrm{p}$

```
#example[$sem(C_(+"pq")) = lam("p", lam("q",
  "q" = "p" or "q" = not "p"))$]
```

Combine a gloss with a denotation in the same example:

(8) Heidi döner-i ye-di=mi?
Heidi döner-ACC eat-PST.3SG=Q
'Did Heidi eat the döner?'
$[\![\ldots]\!] = \left\{ \begin{matrix} \mathrm{ate(heidi,doner)} \\ \neg\ \mathrm{ate(heidi,doner)} \end{matrix} \right\}$

```
#example[
  - Heidi  döner-i       ye-di=mi?
  - Heidi  döner-#acc    eat-#pst.3#smallcaps[sg]=#q
  'Did Heidi eat the döner?'

  $sem("...") = mat(delim: "{",
    "ate"("heidi","doner") ,;
    not "ate"("heidi","doner");
  )$
]
```

4.7 Multi-Line Derivations

Aligned math with `&` and `\`:

(9) $[\![\text{Mary runs}]\!]$

$$\begin{aligned}[\![\text{Mary runs}]\!] &= [\![\text{runs}]\!]([\![\text{Mary}]\!]) \\ &= (\lambda\ \text{x}\ .\ \text{run}(\text{x}))(m) \\ &\underset{\beta}{\Rightarrow} \text{run}(m)\end{aligned}$$

```
#example[$sem("Mary runs")$

  $ sem("Mary runs")
      &= sem("runs")(sem("Mary")) \
      &= (lam("x", "run"("x")))(m) \
      &=>_beta "run"(m) $
]
```

4.8 Derivations with Rule Labels

For right-aligned rule names, use a four-column grid (LHS, operator, RHS, label) so `=` and $\underset{\beta}{\Rightarrow}$ stay vertically aligned:

```
#let derivation(..steps) = {
  set par(first-line-indent: 0em)
  let cells = ()
  for step in steps.pos() {
    let (lhs, op, rhs, label) = step
    cells.push(align(right, lhs))
    cells.push(op)
    cells.push(rhs)
    cells.push(align(right,
      text(size: 9pt, fill: luma(100))[#label]))
  }
  block(inset: (left: 0em, y: 0.3em))[
    #grid(columns: (auto, auto, 1fr, auto),
      column-gutter: 0.4em, row-gutter: 0.45em,
      ..cells)
  ]
}
```

Code

```
#example[$sem("Mary loves John")$

  #derivation(
    ($sem("loves John")$, $=$,
     $(lam("y", lam("x", "love"("x","y"))))(j)$,
     [by FA]),
    ([], $=>_beta$,
     $lam("x", "love"("x", j))$,
     [by $beta$-red.]),
    ($sem("Mary loves John")$, $=$,
     $(lam("x", "love"("x", j)))(m)$,
     [by FA]),
    ([], $=>_beta$,
     $"love"(m, j)$,
     [by $beta$-red.]),
```

```
  )
]
```

Output

(10) $[\![\text{Mary loves John}]\!]$

$$\begin{aligned} [\![\text{loves John}]\!] &= (\lambda\ \text{y}\ .\ \lambda\ \text{x}\ .\ \text{love}(\text{x},\text{y}))(j) && \text{by FA} \\ &\underset{\beta}{\Rightarrow} \lambda\ \text{x}\ .\ \text{love}(\text{x},j) && \text{by } \beta\text{-red.} \\ [\![\text{Mary loves John}]\!] &= (\lambda\ \text{x}\ .\ \text{love}(\text{x},j))(m) && \text{by FA} \\ &\underset{\beta}{\Rightarrow} \text{love}(m,j) && \text{by } \beta\text{-red.} \end{aligned}$$

4.9 Hamblin / Alternative Sets

(11) Polar question Hamblin set:

$$\left\{ \begin{matrix} \text{ate}(\text{heidi},\text{doner}) \\ \neg\ \text{ate}(\text{heidi},\text{doner}) \end{matrix} \right\}$$

(12) Focus alternative set:

$$\left\{ \begin{matrix} \text{ate}(\text{heidi},\text{doner}) \\ \text{ate}(\text{heidi},\text{dolma}) \\ \text{ate}(\text{heidi},\text{donut}) \\ \vdots \end{matrix} \right\}$$

```
#example[Polar question:
  $ mat(delim: "{",
      "ate"("heidi","doner") ,;
      not "ate"("heidi","doner");
    ) $]
```

4.10 Generalized Quantifiers

(13) $[\![\text{every}]\!] = \lambda\ \text{P}\ .\lambda\ \text{Q}\ .\forall x[\text{P}(x) \rightarrow \text{Q}(x)]$
$[\![\text{some}]\!] = \lambda\ \text{P}\ .\lambda\ \text{Q}\ .\exists x[\text{P}(x) \wedge \text{Q}(x)]$
$[\![\text{no}]\!] = \lambda\ \text{P}\ .\lambda\ \text{Q}\ .\neg\exists x[\text{P}(x) \wedge \text{Q}(x)]$

4.11 Type-Lifting

(14) $$\text{Lift} = \lambda\ \text{x}\ .\ \lambda\ \text{P}\ .\ \text{P}(\text{x}) : e \rightarrow ((e \rightarrow t) \rightarrow t)$$

4.12 Answerhood Operators

Dayal's (1996) Ans:

(15) $[\![\text{Ans}]\!](Q) = \lambda\ \text{w}\ .\ \iota p \in Q[p(w) \wedge \forall p' \in Q[p(w) \rightarrow p \subseteq p']]$

4.13 Focus-to-Ordinary C Head

(16) $[\![[C[TP]]]\!]^o = [\![\text{TP}]\!]^f$

(17) $[\![\Sigma_F]\!]^f = \{\lambda\ \mathrm{p}\ .\ \mathrm{p},\quad \lambda\ \mathrm{p}\ .\ \neg\ \mathrm{p}\}$

5 Syntactic Trees

The `syntree` package draws trees from bracket notation:

```
#import "@preview/syntree:0.2.1": syntree, tree
```

5.1 Simple Trees

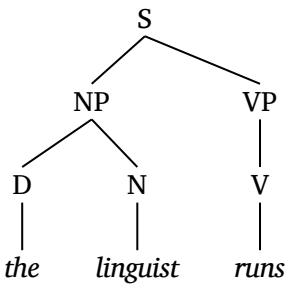
Pass a bracket string to `#syntree()`:

Code

```
#example[
  #syntree(
    nonterminal: (font: "Charis SIL"),
    terminal: (font: "Charis SIL", style: "italic"),
    child-spacing: 2em, layer-spacing: 2.3em,
    "[S [NP [D the] [N linguist]] [VP [V runs]]]"
  )
]
```

Output

(18)



5.2 A Turkish Polar Question Tree

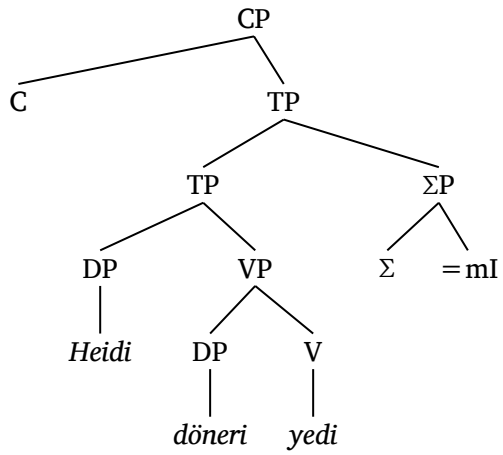
DP and VP merge into a lower TP; this lower TP and ΣP merge into a higher TP; C and the higher TP form CP:

Code

```
#example[
  #syntree(
    "[CP [C] [TP [TP [DP Heidi]
      [VP [DP döneri] [V yedi]]]
      [ΣP [Σ] [=mI]]]]"
  )
]
```

Output

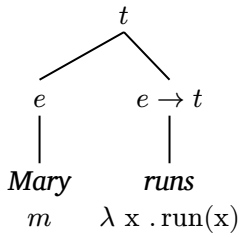
(19)



5.3 Trees with Semantic Types on Nodes

The `tree()` function accepts arbitrary Typst content, so you can put types and denotations directly on nodes:

(20)




```
#example[
  #tree(
    $t$,
    tree($e$, [_Mary_ \ $m$]),
    tree($e -> t$,
      [_runs_ \ $lam("x", "run"("x"))$]),
  )
]
```

5.4 Semantic Derivation on a Tree

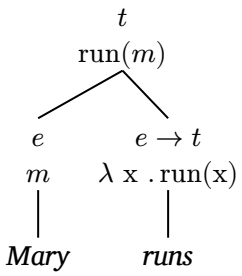
The composed result sits at the mother node:

Code

```
#example[
  #tree(
    [$t$ \ $"run"(m)$],
    tree([$e$ \ $m$], [_Mary_]),
    tree([$e -> t$ \ $lam("x", "run"("x"))$],
      [_runs_]),
  )
]
```

Output

(21)



5.5 Movement

Mark the landing site and trace with subscripts. `syntree` uses `$zws_i$` (zero-width space trick) for subscripts on nonterminals:

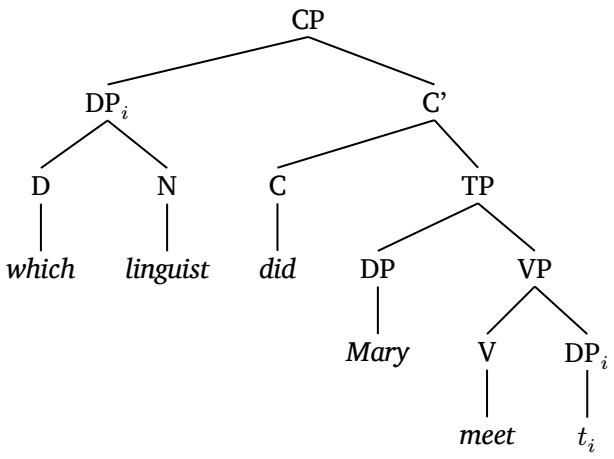
Code

```
#example[_Which linguist did Mary meet?_

  #syntree(
    "[CP [DP$zws_i$ [D which] [N linguist]]
      [C' [C did] [TP [DP Mary]
        [VP [V meet] [DP$zws_i$ $t_i$]]]]]"
  )
]
```

Output

(22) *Which linguist did Mary meet?*



For actual movement **arrows**, use `lingotree` with `mannot` and `cetz`:

Code

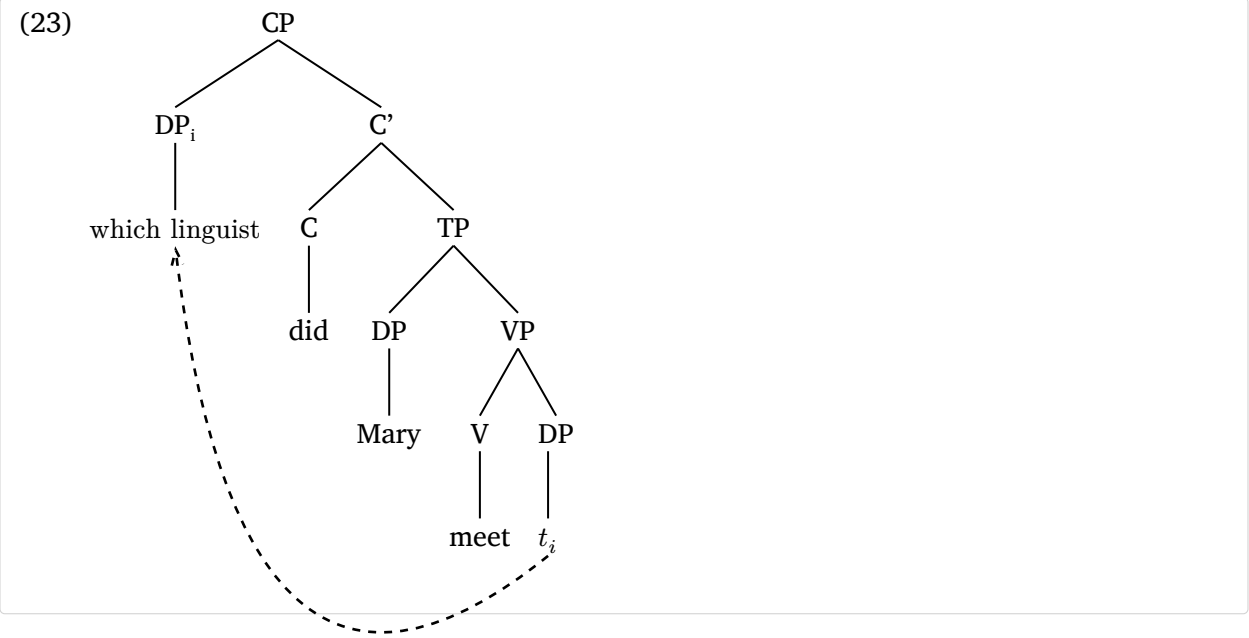
```
#import "@preview/lingotree:1.0.0": tree as ltree, render as lrender
#import "@preview/mannot:0.3.0": mark, annot-cetz
#import "@preview/cetz:0.4.2" as cetz

#example[
  #lrender(
    ltree(
```

```
      tag: [CP],
      ltree(tag: [DP#sub[i]],
        mark(tag: <landing>)[which linguist]),
      ltree(tag: [C'],
        ltree(tag: [C], [did]),
        ltree(tag: [TP],
          ltree(tag: [DP], [Mary]),
          ltree(tag: [VP],
            ltree(tag: [V], [meet]),
            ltree(tag: [DP],
              mark(tag: <trace>)[$t_i$]),
          ),
        ),
      ),
    ),
  )

  #annot-cetz(
    (<trace>, <landing>), cetz,
    {
      cetz.draw.bezier-through(
        "trace.south",
        (rel: (x: -2, y: -1)),
        "landing.south",
        stroke: (dash: "dashed"),
        mark: (end: "straight"),
      )
    },
  )
]
```

Output

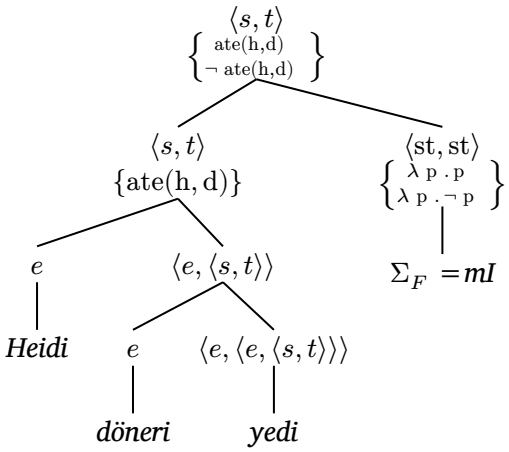


This gives a dashed arrow from the trace to the landing site, like `\draw` in TikZ with `forest`.

5.6 Trees with Hamblin Sets

For alternative-semantics trees, put `mat(delim: "{")` in node labels:

(24)




```
#example[
  #tree(
    [$ty(s,t)$ \ $mat(delim: "{",
        "ate"("h","d") ,;
        not "ate"("h","d");
      )$],
    tree(
      [$ty(s,t)$ \ ${"ate"("h","d")}$],
      tree([$e$], [_Heidi_]),
      tree([$ty(e,ty(s,t))$],
        tree([$e$], [_döneri_]),
        tree([$ty(e,ty(e,ty(s,t)))$], [_yedi_]),
      ),
    ),
    tree(
      [$ty("st","st")$ \ $mat(delim: "{",
          lam("p","p") ,;
          lam("p", not "p");
        )$],
      [$Sigma_F$ _=mI_],
    ),
  )
]
```

6 Quick Reference

What you want	Code
$[\![\text{runs}]\!]$	`$sem("runs")$`
$[\![\text{TP}]\!]^{o}$ / $[\![\text{TP}]\!]^{f}$	`$semo("TP")$ / $semf("TP")$`
$\lambda\ \text{x}\ .\ \varphi(x)$	`$lam("x", phi(x))$`
$e \to t$	`$e -> t$`
$\langle e, t \rangle$	`$ty(e, t)$`
$\forall x[P(x)]$	`$forall x [P(x)]$`
$\exists x[P(x)]$	`$exists x [P(x)]$`
$\varphi[\text{x} \coloneqq \text{a}]$	`$phi["x" := "a"]$`
$(\lambda\ \text{x}\ .\ \varphi) \underset{\beta}{\rightsquigarrow} \varphi$	`~>_beta`
$\Box p$ / $\diamond p$	`$square p$ / $diamond.stroked p$`
$\iota x[P(x)]$	`$iota x [P(x)]$`

Table 2: Quick reference for semantic notation.

Package	Use for
`eggs`	examples, sub-examples, judges, glosses, refs
`leipzig-glossing`	alternative glossing package
`syntree`	simple bracket-notation trees, `tree()` for rich nodes
`lingotree`	trees with styling, colors, annotations
`cetz`	general drawing (arrows, boxes, diagrams)
`mannot`	movement arrows on trees

Table 3: Useful packages for linguistics in Typst.